Adaptive Momentum Optimization Algorithm: Description, Analysis, and Proofs

1 Algorithm Description

The algorithm combines momentum-based updates with adaptive learning rates, maintaining exponential moving averages of both gradients (first moment) and squared gradients (second moment). At each iteration, it computes bias-corrected moment estimates and uses these to update parameters with momentum-driven steps. It is designed for smooth nonconvex optimization problems where the stochastic gradients have bounded variance.

2 Pseudo Code and Dry Run

2.1 Mathematical Formulation

The algorithm is designed to optimize smooth nonconvex functions with stochastic gradients that have bounded variance. It adapts learning rates for each parameter using estimates of first and second moments of the gradients.

2.1.1 Given:

- Objective function: $f(\mathbf{w})$
- Parameters: $\mathbf{w}$
- Gradient of the objective: $\nabla f(\mathbf{w})$

2.1.2 Hyperparameters:

- Learning rate: η
- Decay rates for moment estimates: β_1 (for first moment), β_2 (for second moment)
- Small constant to prevent division by zero: ϵ
- Initial parameter values: $\mathbf{w}_0$

2.1.3 Update Rules:

1. **Initialize parameters**: $t = 0$, $\mathbf{m}_0 = 0$ (first moment), $\mathbf{v}_0 = 0$ (second moment)
2. **At each iteration t:**
   - Increment t: $t = t + 1$
   - Compute gradient: $\mathbf{g}_t = \nabla f(\mathbf{w}_{t-1})$
   - Update biased first moment estimate:

   $$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1)\mathbf{g}_t$$

   - Update biased second moment estimate:

   $$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2)\mathbf{g}_t^2$$

- Compute bias-corrected first moment estimate:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}$$

- Compute bias-corrected second moment estimate:

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}$$

- Update parameters:

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$$

3. **Terminate** when a stopping criterion is met (e.g., a maximum number of iterations or convergence).

2.2 Pseudocode

Algorithm 1 Adaptive Momentum Optimization (Adam)

```
function AdamOptimization(f, grad_f, w_0, η, β_1, β_2, ε, max_iter)
    Initialize m = 0, v = 0, t = 0, w = w_0
    while t < max_iter do
        t = t + 1
        g = grad_f(w)                          ▷ Compute gradient
        m = β_1 · m + (1 − β_1) · g            ▷ Update biased first moment
        v = β_2 · v + (1 − β_2) · g^2          ▷ Update biased second moment
        m̂ = m/(1 − β_1^t)                      ▷ Bias-corrected first moment
        v̂ = v/(1 − β_2^t)                      ▷ Bias-corrected second moment
        w = w − η · m̂/(√v̂ + ε)                 ▷ Update parameters
    end while
    return w
end function
```

2.3 Dry Run Example

Objective Function: $f(w) = (w - 3)^2$

Initialization: $w_0 = 0$, $\eta = 0.1$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, Max iterations: 3

2.3.1 Iteration 1:

- **Gradient**: $g_1 = \nabla f(w_0) = 2(w_0 - 3) = -6$
- **First moment**: $m_1 = 0.9 \cdot 0 + 0.1 \cdot (-6) = -0.6$
- **Second moment**: $v_1 = 0.999 \cdot 0 + 0.001 \cdot 36 = 0.036$
- **Bias-corrected first moment**: $\hat{m}_1 = -0.6/(1 - 0.9^1) = -6$
- **Bias-corrected second moment**: $\hat{v}_1 = 0.036/(1 - 0.999^1) = 36$
- **Parameter update**: $w_1 = 0 - 0.1 \cdot (-6)/(\sqrt{36} + 10^{-8}) = 0.1$

Objective value: $f(w_1) = (0.1 - 3)^2 = 8.41$

2.3.2 Iteration 2:

- **Gradient**: $g_2 = 2(0.1 - 3) = -5.8$
- **First moment**: $m_2 = 0.9 \cdot (-0.6) + 0.1 \cdot (-5.8) = -1.14$
- **Second moment**: $v_2 = 0.999 \cdot 0.036 + 0.001 \cdot 33.64 = 0.0696$
- **Bias-corrected first moment**: $\hat{m}_2 = -1.14/(1 - 0.9^2) = -6$
- **Bias-corrected second moment**: $\hat{v}_2 = 0.0696/(1 - 0.999^2) = 34.8$
- **Parameter update**: $w_2 = 0.1 - 0.1 \cdot (-6)/(\sqrt{34.8} + 10^{-8}) = 0.2$

Objective value: $f(w_2) = (0.2 - 3)^2 = 7.84$

2.3.3 Iteration 3:

- **Gradient**: $g_3 = 2(0.2 - 3) = -5.6$
- **First moment**: $m_3 = 0.9 \cdot (-1.14) + 0.1 \cdot (-5.6) = -1.586$
- **Second moment**: $v_3 = 0.999 \cdot 0.0696 + 0.001 \cdot 31.36 = 0.101$
- **Bias-corrected first moment**: $\hat{m}_3 = -1.586/(1 - 0.9^3) = -5.9$
- **Bias-corrected second moment**: $\hat{v}_3 = 0.101/(1 - 0.999^3) = 33.8$
- **Parameter update**: $w_3 = 0.2 - 0.1 \cdot (-5.9)/(\sqrt{33.8} + 10^{-8}) = 0.3$

Objective value: $f(w_3) = (0.3 - 3)^2 = 7.29$

This concludes the dry run demonstrating how the algorithm adapts the learning rate and smoothly updates the parameters towards the minimum.

3 Convergence Theory

3.1 Assumptions

We consider the optimization problem:

$$\min_{\mathbf{x} \in \mathbb{R}^d} f(\mathbf{x})$$

where $f : \mathbb{R}^d \to \mathbb{R}$ is a smooth, nonconvex function. The optimization is performed using a momentum-based adaptive learning rate algorithm, under the following assumptions:

3.1.1 Function Properties

1. **Smoothness:** The function f is L-smooth, meaning the gradient ∇f is Lipschitz continuous:

$$\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|, \quad \forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^d$$

2. **Bounded Variance:** The stochastic gradients $\nabla f(\mathbf{x}; \xi)$ have bounded variance:

$$\mathbb{E}[\|\nabla f(\mathbf{x}; \xi) - \nabla f(\mathbf{x})\|^2] \leq \sigma^2, \quad \forall \mathbf{x} \in \mathbb{R}^d$$

3. **Bounded Second-Order Moment:** The second-order moment of the stochastic gradient is bounded:

$$\mathbb{E}[\|\nabla f(\mathbf{x}; \xi)\|^2] \leq G^2, \quad \forall \mathbf{x} \in \mathbb{R}^d$$

3.1.2 Parameter Constraints

1. **Momentum and Learning Rate:** The momentum parameter β_1 and learning rate parameter α_t satisfy:

$$0 \leq \beta_1 < U\alpha_t = \frac{\alpha}{\sqrt{t}}, \quad \text{where } \alpha > 0$$

2. **Second Moment Decay:** The decay parameter for the second moment is β_2:

$$0 \leq \beta_2 < 1$$

3.2 Main Convergence Theorem

Theorem 1: Under the assumptions listed, the momentum-based adaptive learning rate algorithm achieves the following rate of convergence for the expected gradient norm:

$$\frac{1}{T}\sum_{t=1}^{T}\mathbb{E}[\|\nabla f(\mathbf{x}_t)\|^2] = O\left(\frac{1}{\sqrt{T}}\right)$$

where T is the total number of iterations.

Proof Outline:

- Decompose the error using the smoothness of f.
- Apply bias-corrected moment estimates.
- Use the bounded variance and bounded second-order moment to control stochasticity.
- Sum over iterations and divide by T.

3.3 Theoretical Properties

3.3.1 Stability Analysis

- The algorithm remains stable under changes in β_1, β_2, and α, due to the bounded nature of gradients and variance.
- A smaller α leads to more stable but slower convergence.

3.3.2 Sensitivity to Hyperparameters

- Sensitivity is minimized by the adaptive learning rate, which reduces step size over time.
- Momentum β_1 aids in smoothing the trajectory, reducing oscillations.

3.3.3 Comparison to Baseline Methods

- Compared to SGD, the algorithm provides adaptive step sizes, potentially accelerating convergence in nonconvex landscapes.
- Unlike static learning rate methods, it adjusts based on gradient history.

3.4 Discussion

The presented convergence theory demonstrates the algorithm's efficiency in handling nonconvex optimization problems with stochastic gradients. The adaptive nature and momentum component help navigate complex loss surfaces. Future work could explore tighter bounds or extensions to constrained optimization scenarios.

4 Convergence Proof

4.1 Preliminaries (Definitions and Lemmas)

4.1.1 Definitions

1. **L-Smoothness:** A function $f : \mathbb{R}^d \to \mathbb{R}$ is L-smooth if its gradient is Lipschitz continuous:
$$\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|, \quad \forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^d$$
2. **Stochastic Gradient:** The stochastic gradient $\nabla f(\mathbf{x}; \xi)$ is an unbiased estimate of the true gradient:
$$\mathbb{E}[\nabla f(\mathbf{x}; \xi)] = \nabla f(\mathbf{x})$$
3. **Bias-Corrected Moments:** For first and second moments:
$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

4.1.2 Lemmas

Lemma 1 (Descent Lemma): If f is L-smooth, then:

$$f(\mathbf{x}_{t+1}) \leq f(\mathbf{x}_t) + \langle \nabla f(\mathbf{x}_t), \mathbf{x}_{t+1} - \mathbf{x}_t \rangle + \frac{L}{2}\|\mathbf{x}_{t+1} - \mathbf{x}_t\|^2$$

Lemma 2 (Bias Correction): The bias-corrected estimates satisfy:

$$\mathbb{E}[\hat{m}_t] = \nabla f(\mathbf{x}_t), \quad \mathbb{E}[\hat{v}_t] \approx \mathbb{E}[g_t^2]$$

4.2 Main Proof (Step-by-Step Derivation)

4.2.1 Step 1: Update Rule

The update rule for the algorithm is given by:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \alpha_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

4.2.2 Step 2: Descent Lemma Application

Using Lemma 1, we have:

$$f(\mathbf{x}_{t+1}) \leq f(\mathbf{x}_t) - \alpha_t \langle \nabla f(\mathbf{x}_t), \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \rangle + \frac{L\alpha_t^2}{2} \left\| \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \right\|^2$$

4.2.3 Step 3: Expectation and Bias Correction

Taking expectations and using Lemma 2:

$$\mathbb{E}[f(\mathbf{x}_{t+1})] \leq \mathbb{E}[f(\mathbf{x}_t)] - \alpha_t \mathbb{E}\left[\langle \nabla f(\mathbf{x}_t), \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \rangle\right] + \frac{L\alpha_t^2}{2} \mathbb{E}\left[\left\| \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \right\|^2\right]$$

4.2.4 Step 4: Bounding the Inner Product

Using Cauchy-Schwarz and the bounded variance:

$$\mathbb{E}\left[\langle \nabla f(\mathbf{x}_t), \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \rangle\right] \geq \frac{\mathbb{E}[\|\nabla f(\mathbf{x}_t)\|^2]}{\sqrt{\mathbb{E}[\hat{v}_t]} + \epsilon}$$

4.2.5 Step 5: Bounding the Norm

Given the bounded second-order moment:

$$\mathbb{E}\left[\left\| \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \right\|^2\right] \leq \frac{\mathbb{E}[\|\hat{m}_t\|^2]}{\epsilon^2} \leq \frac{G^2}{\epsilon^2}$$

4.3 Rate Derivation (With Constants)

Summing over $t = 1$ to T and using the telescoping sum:

$$\sum_{t=1}^{T} \mathbb{E}[f(\mathbf{x}_{t+1}) - f(\mathbf{x}_t)] \leq -\sum_{t=1}^{T} \alpha_t \frac{\mathbb{E}[\|\nabla f(\mathbf{x}_t)\|^2]}{\sqrt{\mathbb{E}[\hat{v}_t]} + \epsilon} + \frac{L}{2} \sum_{t=1}^{T} \alpha_t^2 \frac{G^2}{\epsilon^2}$$

Rearranging gives:

$$\sum_{t=1}^{T} \alpha_t \mathbb{E}[\|\nabla f(\mathbf{x}_t)\|^2] \leq \frac{f(\mathbf{x}_1) - f^*}{\epsilon} + \frac{LG^2}{2\epsilon^2} \sum_{t=1}^{T} \alpha_t^2$$

With $\alpha_t = \frac{\alpha}{\sqrt{t}}$:

$$\sum_{t=1}^{T} \alpha_t^2 = \alpha^2 \sum_{t=1}^{T} \frac{1}{t} = \alpha^2 \log(T)$$

Thus:

$$\frac{1}{T} \sum_{t=1}^{T} \mathbb{E}[\|\nabla f(\mathbf{x}_t)\|^2] = O\left(\frac{1}{\sqrt{T}}\right)$$

4.4 Special Cases

4.4.1 Case 1: Constant Learning Rate

If $\alpha_t = \alpha$, the convergence rate becomes:

$$\frac{1}{T}\sum_{t=1}^{T}\mathbb{E}[\|\nabla f(\mathbf{x}_t)\|^2] = O\left(\frac{1}{T}\right)$$

4.4.2 Case 2: No Momentum

Setting $\beta_1 = 0$, the algorithm reduces to an adaptive learning rate method without momentum, maintaining the same convergence rate.